

Synthetic data evaluation

B2B paper

Synthetic data encode ground truth as a binary causal-selection structure, 1/0 causal labels

Model coefficients: B2B, Forward, Backward, CCA, PLS

B2B	Forward	Backward	CCA & PLS
$\hat{S} \in \mathbb{R}^{d_x}$	$H_i \in \mathbb{R}^{d_y}$	$G_i \in \mathbb{R}^{d_y}$	$H_i \in \mathbb{R}^{d_z}$
Scalar coefficients	Loading vectors, take the sum of squared coefficients: $\hat{S}_i = \sum_j H_i^{j^2}$		

AUC(S, $\hat{S}$): evaluates whether truly causal predictors are ranked above non-causal predictors.

Limitation: ranking metric only, it does not directly quantify exact feature recovery.

Unfold Simulation

Ground truth is defined as predictor-specific waveforms (e.g., P300- and N170-like)

Model coefficients: Backward, rERP, B2B, rERP+B2B

Backward sanity check	rERP	B2B	rERP+B2B
$G_i(t) \in \mathbb{R}^{d_y}$	$\hat{\beta}_i(\cdot) \in \mathbb{R}^{d_t}$	$\hat{S}_i(\cdot) \in \mathbb{R}^{d_t}$	$\hat{S}_i^{oc}(\cdot) \in \mathbb{R}^{d_t}$
Decoding-weights (excluded from waveform-recovery comparisons)	Recovered waveform	Predictor-specific waveform	Overlap-corrected predictor-specific waveform

Waveform recovery:

Time-course correlation: $r(\hat{w}_i(t), w_i^{true}(t))$

$$RMSE_i = \sqrt{\frac{1}{T} \sum_t (\hat{w}_i(t) - w_i^{true}(t))^2}$$

Cross-talk matrix: $M_{ij} = \text{corr}(\hat{w}_i(t), w_j^{true}(t))$

Conceptual structure of B2B: outer evaluation vs inner training

Algorithm 2: B2B feature importance.

Input: $X_{train} \in \mathbb{R}^{m \times d_x}$, $X_{test} \in \mathbb{R}^{m' \times d_x}$, $Y_{train} \in \mathbb{R}^{m \times d_y}$, $Y_{test} \in \mathbb{R}^{m' \times d_y}$,

Output: estimate of prediction improvement $\Delta R \in \mathbb{D}^{d_x}$.

```

1  $H, G = \text{B2B}(X_{train}, Y_{train});$ 
2  $R_{full} = \text{corr}(X_{test}H, Y_{test}G);$ 
3 for  $i = 1, \dots, d_x$  do
4    $K = Id;$ 
5    $K[i] \leftarrow 0;$ 
6    $R_k = \text{corr}(X_{test}KH, Y_{test}G_i);$ 
7    $\Delta R_i = R_{full} - R_k;$ 
8 end
9 return  $\Delta R$ 

```

Inner loop: B2B training

- Split training data into two subsets
- Fit G : $Y_1 \rightarrow X_1$
- Fit H : $X_2 \rightarrow Y_2 \hat{G}$
- Ensemble over random splits
- Output: $G, H, \text{diag}(H)$

Outer loop: held-out feature importance

- Fit on train
- Evaluate on test
- Compare full vs knockout
- Output: ΔR_i

Algorithm 1: Back-to-back regression.

Input: input data $X \in \mathbb{R}^{m \times d_x}$, output data $Y \in \mathbb{R}^{m \times d_y}$, number of repetitions $m \in \mathbb{N}$.

Output: estimate of causal influences $\hat{S} \in \mathbb{D}^{d_x \times d_x}$.

```

1  $\hat{S} \leftarrow 0;$ 
2 for  $i = 1, \dots, m$  do
3    $(X, Y) \leftarrow \text{ShuffleRows}((X, Y));$ 
4    $(X_1, Y_1), (X_2, Y_2) \leftarrow \text{SplitRowsInHalf}((X, Y));$ 
5    $\hat{G} = \text{LinearRegression}(Y_1, X_1);$ 
6    $\hat{H} = \text{LinearRegression}(X_2, Y_2 \hat{G});$ 
7    $\hat{S} \leftarrow \hat{S} + \text{diag}(\hat{H});$ 
8 end
9  $\hat{S} \leftarrow \hat{S}/m;$ 
10  $\hat{W} \leftarrow \text{LinearRegression}(X \hat{S}, Y);$ 
11 return  $\hat{S}, \hat{W}$ 

```

$$\triangleright \hat{G} = (Y_1^\top Y_1 + \Lambda_Y)^{-1} Y_1^\top X_1$$

$$\triangleright \hat{H} = (X_2^\top X_2 + \Lambda_X)^{-1} X_2^\top Y_2 \hat{G}$$

$$\hat{S} = \text{diag}(\underbrace{(X_2^\top X_2 + \Lambda_X)^{-1} X_2^\top Y_2 (Y_1^\top Y_1 + \Lambda_Y)^{-1} Y_1^\top X_1}_{\substack{1) \hat{X}: \text{regression from } Y \text{ to } X \\ 2) \text{regression from } X \text{ to } \hat{X}}})$$

Held-out feature-importance procedure

1. Split data into train and test
2. Fit full B2B model on train
3. Compute full score on test

```
for train, test in cv.split(X, Y):  
    model.fit(X[train], Y[train])  
    r[t] += score_knockout(model, X[test], Y[test])
```

4. For each feature f :

- knock out feature f in train and test

- keep G fixed, refit H on knockout train data

- compute knockout score on test

- $\Delta R_f = R_{full} - R_{knockout_f}$

```
def score_knockout(model, X, Y, XY_train=None, scoring='r', fix_grid=True):
```

```
    # Compute knock out scores
```

```
    for f in range(dim_x):
```

```
        # Setup knockout matrix
```

```
        knockout = np.eye(dim_x) # np.eye creates an identity matrix of size dim_x
```

```
        knockout[f] = 0
```

```
        model_ = model
```

```
        # refit the model
```

```
        model_ = deepcopy(model)
```

```
        if is_b2b:
```

```
            model_.fit_H(X_train @ knockout, Y_train)
```

```
        # Score
```

```
        score_ko = model_.score(X @ knockout, Y,  
                                scoring=scoring,  
                                multioutput='raw_values')
```

```
class B2B(BaseEstimator, RegressorMixin):
```

```
    def fit_H(self, X, Y):
```

```
        assert hasattr(self, 'G_')
```

```
        YG = Y @ self.G_.T
```

```
        # Fit encoder
```

```
        if self.H == 'ridge_cv':
```

```
            self.H_, H_alpha, _ = ridge_cv(X, YG,  
                                             self.alphas,  
                                             self.independent_alphas)
```

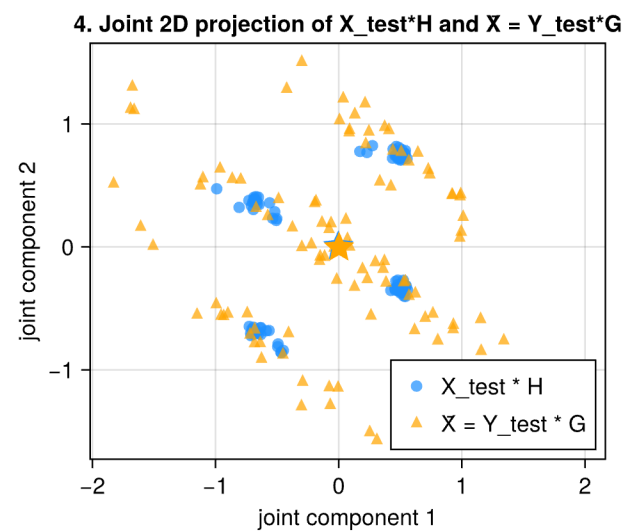
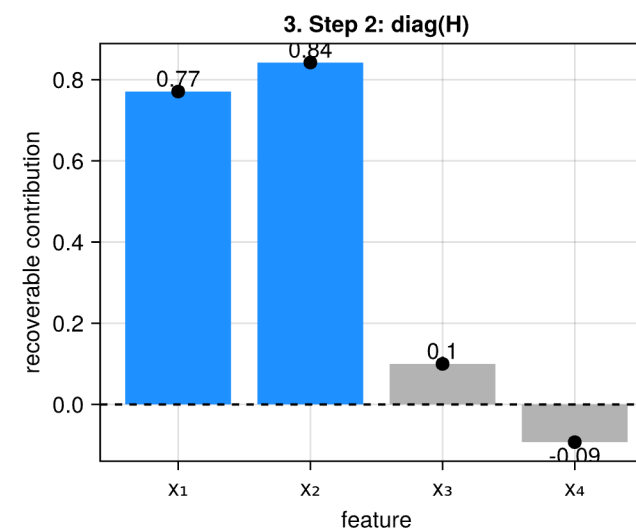
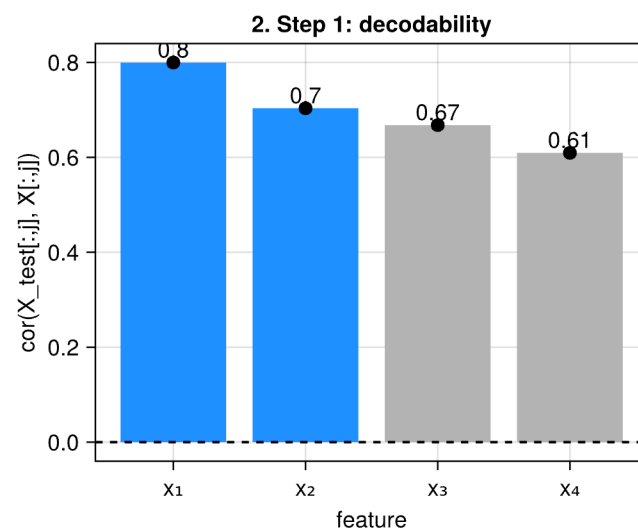
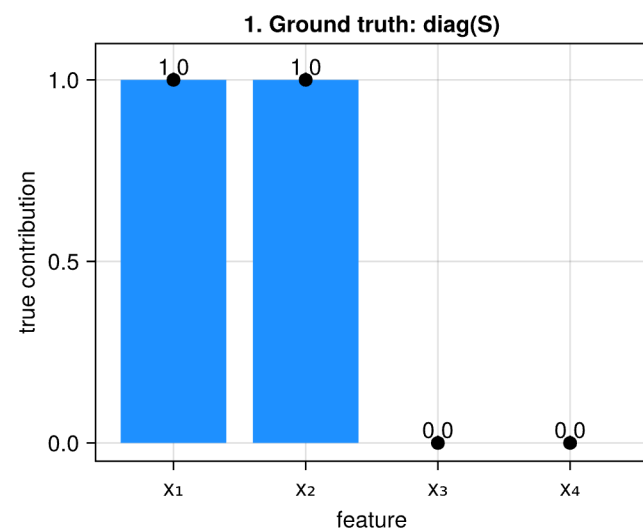
```
        else:
```

```
            raise NotImplementedError
```

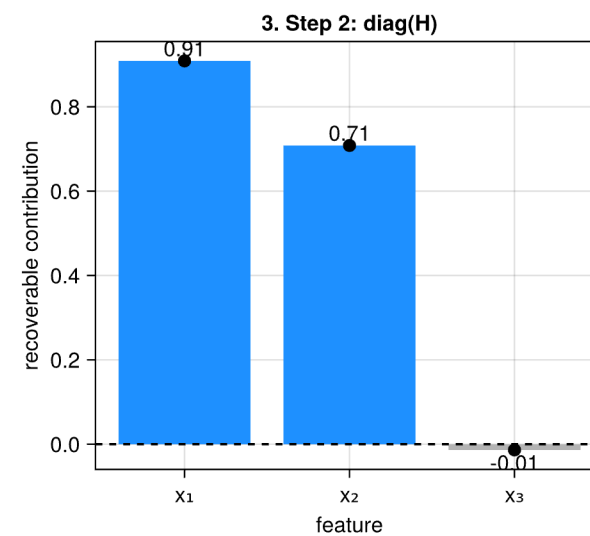
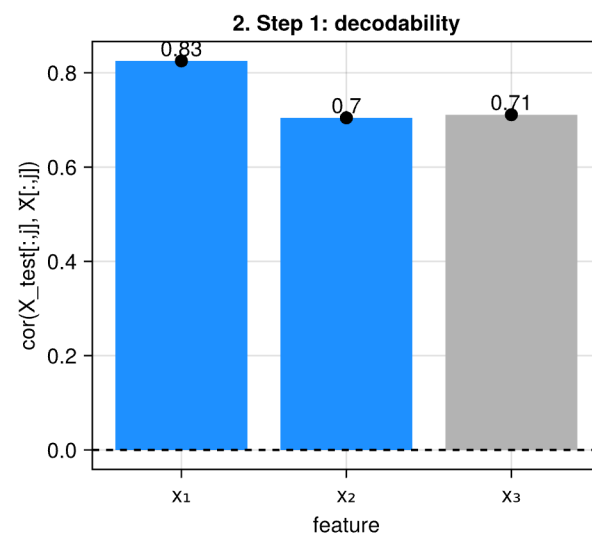
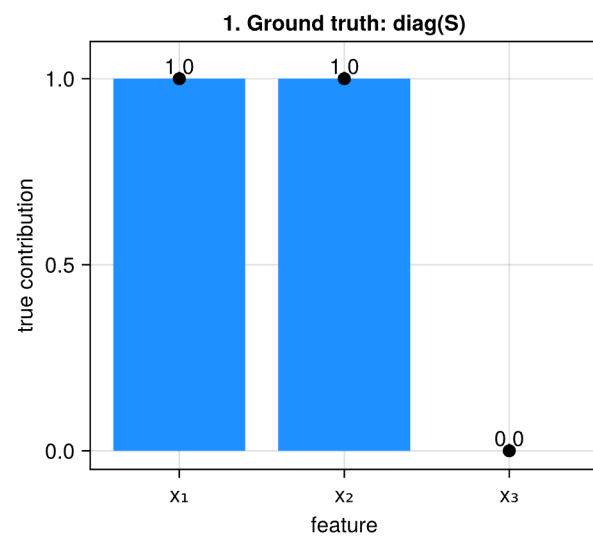
```
        # Aggregate ensembling
```

```
        self.S_ = np.diag(self.H_)
```

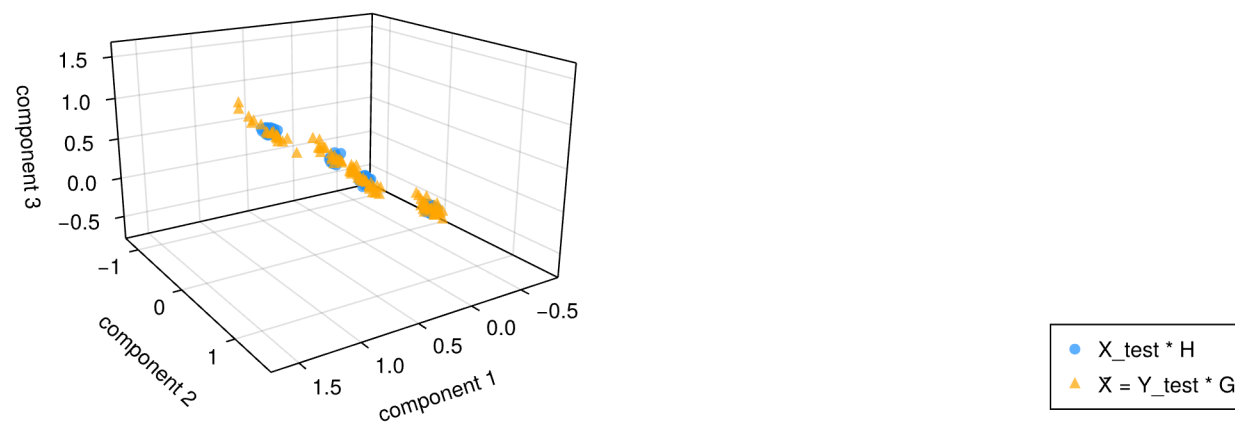
```
        return self
```



Top row: true contribution → what is decodable → what B2B recovers. Bottom: blue = $X_{\text{test}}*H$, orange = $X = Y_{\text{test}}*G$.



4. 3D comparison: $X_{\text{test}}*H$ vs $X = Y_{\text{test}}*G$



Top row: true contribution → what is decodable → what B2B recovers. Bottom: blue = $X_{\text{test}}*H$, orange = $X = Y_{\text{test}}*G$.